



PLAYING OWARE NAM-NAM WITH DEEP Q-NETWORKS

Lappeenranta-Lahti University of Technology LUT

Master's Program in Computational Engineering, Master's Thesis

2024

Chantelle Amoako-Atta

Examiners: Academy Research Fellow Andreas Rupp
 Professor Heikki Haario

ABSTRACT

Lappeenranta-Lahti University of Technology LUT
School of Engineering Science
Computational Engineering

Chantelle Amoako-Atta

Playing Oware Nam-nam with Deep Q-Networks

Master's thesis

2024

43 pages, 10 figures, 3 appendices

Examiners: Academy Research Fellow Andreas Rupp and Professor Heikki Haario

Keywords: deep Q-learning, double deep Q-learning, oware, machine learning, model-free learning, nam-nam, reinforcement learning

In this thesis, deep Q-learning agents are used to learn how to play “Nam-Nam” Oware, a game known for its complex and unpredictable nature. We use Deep Q-Networks (DQN) and Double Deep Q-Networks (DDQN) in the study to simulate realistic gameplay. Techniques like self-play, epsilon-greedy strategy, and reward shaping are key to enhancing agent performance. The findings show that DDQN agents outperform DQN and random-play agents and produce more stable results. Our sensitivity analysis of hyperparameters further shows their influence on learning. This work not only advances reinforcement learning (RL) in strategic games but also demonstrates its potential to tackle real-world decision-making challenges that seem to be more random than strategic.

ACKNOWLEDGEMENTS

I would like to thank LUT University, African Institute for Mathematical Sciences (AIMS-NEI Initiative) and (Quantum Leap Africa) QLA for making it possible for me to study and produce this work at LUT university through their full-funding of my masters programme. I also extend immense gratitude to my lecturers and supervisors, who have been an truly valuable source of knowledge and inspiration for me, and finally I would like to duly acknowledge all the academics, through whose hard work and dedication reinforcement learning as we know it, and its application in gameplay is at this remarkable level of progress, and from whose works I have learned and applied in this thesis, who have also been duly acknowledged in the references.

Chantelle Amoako-Atta

LIST OF ABBREVIATIONS

RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
DQN	Deep Q-Network
DDQN	Double Deep Q-Network
MDP	Markov Decision Process
AI	Artificial Intelligence
A3C	Asynchronous Advantage Actor Critic
SARSA	State-Action-Reward-State-Action
DNN	Deep Neural Network
CNN	Convolutional Neural Network
DCNN	Deep Convolutional Neural Network

CONTENTS

1	INTRODUCTION	6
1.1	Background	6
1.2	Objectives	9
1.3	Structure of the Thesis	9
2	LITERATURE REVIEW	10
2.1	Reinforcement Learning: Pre- and Post-DQN	10
2.2	The Impact of DQNs on Gaming	10
2.3	Key Reinforcement Learning Concepts	12
3	PROPOSED METHODS	18
3.1	Deep Q-Networks (DQN)	18
3.2	Double Deep Q-Networks (DDQN)	19
4	EXPERIMENTS	21
4.1	Game Environment Setup	21
4.2	Training	24
4.3	Testing and Evaluation	34
5	CONCLUSION	36
	REFERENCES	38
	APPENDICES	
0.1	Workflow Overview	41
0.2	Game Logic Implementation	42
0.3	State Space Calculation	43

1 INTRODUCTION

1.1 Background

Oware, a well-known two-player board game that is particularly significant in African countries, presents itself in many variants, two of which are: the *Abapa* and *Nam-Nam* variants [1], which are most common in Ghana. Oware *Abapa*, often seen in international competitions, differs in its gameplay style from *Nam-Nam*, which is derived from the *Twɛ* dialect meaning *to roam* [2]. This name was given because it embodies a seemingly random and wandering nature in its strategic play. While this feature of the game diminishes its international competitive appeal, it presents a fascinating subject for exploration in reinforcement learning (RL). A lot of the terms used to describe the actions in the game communicate the concept of farming, as the game was invented during a time when such activities were the prevalent occupation [1].



Figure 1. The traditional oware board (Retrieved from: [3])

Oware *Nam-Nam* is played on a board with 12 pits and 48 seeds [2]. The board is divided into two rows of six pits each, and the players sit on either side of the board. The row of pits closest to a player is their territory. Figure 1 shows the board and two players represented as A and B.

The game starts with 4 seeds in each pit. The first player is usually chosen either randomly or by an agreed method, such as tossing a coin. Suppose that player A goes first, then player A picks up all the seeds from any pit in their territory and sows them separately

B	4	4	4	4	4	4
A	4	4	4	4	4	4



Figure 2. Oware board structure with number of seeds

and in succession individually in a counter-clockwise direction, starting from the next pit. If the last seed lands in an empty pit, player A's turn ends, otherwise, all the seeds located where the last seed landed are picked up by player A, and the sowing process continues until an empty pit is reached [2].

Figure 3 shows the board after player A's first turn. It takes four turns for player A to finish their first round, regardless of which pit they start from. The last seed of the round always lands in the first pit that was emptied at the beginning of the turn.

Start of the game						
B	4	4	4	4	4	4
A	4	4	4	4	4	4



After Player A's first move						
B	4	4	4	4	4	4
A	0	5	5	5	5	4



After Player A's second move						
B	4	4	4	5	5	5
A	0	5	5	5	0	5



After Player A's third move						
B	5	5	5	0	5	5
A	1	5	5	5	0	5



Figure 3. Oware board before and after Player A plays once

The game becomes more complex after the first round, as the board state depends on where player B decides to start their turn. The objective of the game is for a player to be able to capture more seeds than their opponent over several rounds of play. Seeds are captured when a player sows a seed that makes a group of four seeds in any pit on their territory. The player then takes all four seeds and removes them from the board. If the last seed of a turn makes a group of four seeds on the opponent's territory, the player can also capture those seeds. Captured seeds are stored separately and counted at the end of the round [2]. Some oware boards have special pits at the ends of the rows for this purpose, as can be seen in the first picture in Figure 1.

A round ends when all the seeds in the game have been captured. When there are only eight seeds left on the board, the first player to capture four seeds also gets the remaining four seeds. The winner of the round is the person who has captured the largest number of seeds during the round. In the case where, say, player A captures more seeds than the opponent, player A is said to have captured one territory (pit) of the opponent. This means that in the subsequent rounds, the territory of Player A is all the pits in front of him, plus one pit (usually starting from the rightmost pit of the opponent) of the opponent whose territory has been captured. The game ends when one player wins six rounds more than the opponent, meaning the player has captured all six territories of the opponent. This can take a minimum of six rounds but usually takes much longer when both players capture each other's territories continuously.

Many factors affect the complexity of a game's implementation in reinforcement learning (RL), such as the size and structure of the state and action spaces, the dynamics and stochasticity of the game environment, the reward function and its sparsity or delay, the presence and behavior of other agents or opponents, the observability and partiality of the game state, the need for exploration and exploitation, the generalization and transferability of the learned policies, and the computational and memory requirements of the RL algorithm [4].

In the case of Oware *Nam-Nam*, the structure of the state space is such that it is determined by the number of seeds in each of the 12 pits (6 per player). Each pit can have 0 to 48 seeds (the total number of seeds in the game). However, not all distributions are possible due to the game's rules. For instance, it's unlikely for all seeds to end up in a single pit. The state space can be estimated by the number of ways to distribute seeds across 12 pits, which is a problem of distributing n indistinguishable items into k distinguishable bins. The action space also consists of all the possible pits from which a player can start playing. This action space can be represented by all integers, from 1 to 12, however, the number of possible actions a player can take changes over the entire game for both players.

Oware has a deterministic environment since the environment changes only according to the movements made by the players. It also has a non-episodic or sequential environment since any action taken in the present directly affects the future. Oware also has an observable environment, since players can see what the state of the board is at any given point in time, and can determine what it will be like after their opponent takes a turn playing. Oware has a static environment since the environment of the game does not change while a player takes an action. The environment of Oware is discrete, since the number of possible states on the board is finite, although it is a considerably large number.

1.2 Objectives

This research focuses on the application of reinforcement learning in playing Oware *Nam-Nam*. The specific objectives of the study are highlighted below.

- We present a bot that plays and beats a human and a random player at the game Oware *Nam-Nam*
- We explore the seemingly random nature of winning in Oware *Nam-Nam* and gain insights into the significance of some hyperparameters of Deep Q-Networks in the process.

1.3 Structure of the Thesis

In the introduction chapter (1), the groundwork for the thesis is laid with a brief introduction to the Oware game, RL, and the reason RL is being used for Oware *Nam-Nam*. The literature review chapter (2) explores the existing literature on RL and its applications, especially in games, highlighting where Oware fits in this context. Details of the RL algorithms and models developed for Oware are presented in the methodology chapter (3), after which come the applications of these RL methods to the game, and a comparison of the performance with human players is done in the experiments chapter (4). The findings from the previous chapter and their implications are analyzed and discussed in the concluding chapter (5) as well as a summary of the main contributions and future research in the field of RL and Oware.

2 LITERATURE REVIEW

2.1 Reinforcement Learning: Pre- and Post-DQN

Reinforcement learning (RL) is a subfield of machine learning that also exists and is relevant in several different disciplines including control theory, game theory, and statistics where an agent, situated in an environment, learns to take actions and receive rewards from them to the end of helping the agent to learn to act in a certain way when faced with different scenarios within the environment [5]. Markov Decision Processes (MDPs) are a common way to formulate RL mathematically since the goal of the process is to learn how to attain the maximum expected cumulative reward over time. Before deep learning came onto the machine learning scene, RL methods such as Q-learning and State-Action-Reward-State-Action (SARSA) used table-based approaches or simple linear function approximators to estimate action values [6]. This made them better suited to lower-dimensional state space RL problems, as compared to those with high-dimensional inputs [7]. Through DL, a drastic change in RL applications came about, starting from the year 2006, with Geoffrey Hinton's "Deep Belief Networks" [8]. Not too long afterward, an even more startling result of DL in RL, called Deep Q-networks was introduced, following the outstanding results produced by AlexNet, a CNN architecture introduced by Alex Krizhevsky in the ImageNet competition in 2012 [9]. DQNs were created as a result of a combination of the Q-learning method for RL with deep convolutional neural networks (DCNNs), which made it possible for simple linear function approximators to effectively handle high-dimensional data [10], by effectively projecting them into a lower but representative space. DQNs soon showed remarkable results in playing Atari 2600 games, only considering their pixel data as input and learning the best actions to win the games through it, and attaining a super-human level of play for a significant number of the games [11]. Not long afterward, the paper "Human-level control through deep reinforcement learning" [7] was published, further highlighting the capabilities of DQNs to learn effective policies in complex environments.

A closer look at the impact of DQNs on the use of RL for gameplay is considered below.

2.2 The Impact of DQNs on Gaming

One of the earliest successes in applying RL to traditional board games was TD-Gammon, which used temporal difference learning to train a neural network to play Backgammon

at an expert level [12]. TD-Gammon learned strategies through self-play, continually improving its evaluation of board states. Backgammon involves elements of chance, with dice rolls influencing the game state, introducing stochasticity that is absent in many other board games. This randomness requires RL algorithms to accommodate probabilistic outcomes and strategies, differing from the deterministic nature of many other games, including Oware Nam-nam [12].

Another set of games where RL produced outstanding results was the playing of Atari games [7, 13]. Atari games, before the introduction of DL in RL, were difficult to play with RL because of their significantly high-dimensional input space. RL algorithms, particularly deep Q-networks (DQN), when they were introduced, were able to not just learn to play, but also to attain superhuman-level of play on these games, using just the pixel information in the learning process [11]. These outstanding results most importantly, were not just applicable to one game, but they cut across a wide range of the Atari games, demonstrating the versatility and robustness of RL algorithms in mastering gameplay in different gaming environments [7]. Notably, a DQN agent created by DeepMind surpassed human-level performance on a suite of 49 Atari games, illustrating the potential of deep learning in RL.

AlphaGo is another notable RL bot developed by DeepMind, which made history by defeating Lee Sedol, the world champion Go player, in 2016 [14]. The agent was trained with a combination of deep neural networks and RL techniques such as deep Q-learning and Monte Carlo Tree Search (MCTS). AlphaGo’s architecture included policy networks to suggest moves and value networks to predict the winner from a given board position. Importantly, it learned from human expert games, which allowed it to anticipate master-level moves and make optimal decisions [14].

Following AlphaGo’s success, AlphaZero emerged in 2017 as a more generalized algorithm capable of mastering multiple board games, including chess and shogi, without any prior human knowledge other than the game rules [15]. AlphaZero used self-play, continually improving by playing against older versions of itself [16]. This approach enabled AlphaZero to surpass human-level play across different games, demonstrating its flexibility and generalization capabilities [17].

RL has also been applied to Shogi, a complex board game similar to chess but with additional intricacies such as piece drops [17]. Algorithms like MCTS, combined with deep neural networks, have been used to tackle Shogi. For instance, AlphaZero’s success in Shogi was achieved using the same principles applied to chess and Go, involving self-

play and policy/value network training. Shogi’s unique challenges, including its larger state space and strategic depth introduced by piece drops, provide a rich testing ground for RL algorithms [15].

Oware Abapa, a variant of the Oware game, is more widely recognized and less complex than Oware Nam-nam. It is often used as a reference in academic research due to its relative simplicity and standardized rules [18]. Research on Oware Abapa has utilized various RL techniques, including MCTS and Q-learning. These studies typically involve creating digital versions of the game and training agents through self-play or supervised learning from expert games. Compared to Oware Nam-nam, Oware Abapa involves a smaller state space with fewer possible moves and simpler capture mechanics, allowing RL algorithms to achieve effective performance more quickly and with less computational effort [18]. The strategies in Oware Abapa focus on maximizing captures and ensuring that the opponent’s pits remain empty, aligning well with the goals of many RL algorithms [18].

The reviewed works highlight the diverse applications and adaptability of RL in mastering various games, each with unique challenges and mechanics. AlphaGo and AlphaZero demonstrated the power of self-play and deep learning in complex board games, while Atari games showcased RL’s ability to handle diverse and dynamic environments. Shogi and Backgammon presented additional layers of complexity, with piece drops and stochastic elements, respectively. Oware Abapa provided a simpler, yet relevant, context for RL application in traditional board games [18].

In studying Oware Nam-nam, insights can be drawn from these examples. Unlike Backgammon, Oware Nam-nam is deterministic; it has strategic depth, and the game is played in turns, similar to Go and Shogi. It, however, also possesses a unique state space due to the rules of the game, which requires RL algorithms that can learn and handle these sowing and capturing strategies effectively. The insights and ideas obtained from these are applied in the implementation of RL for learning to play Oware Nam-nam [18].

2.3 Key Reinforcement Learning Concepts

Agent

The agent is the learner in reinforcement learning, that interacts with the environment [19]. The agent, based on a particular policy takes actions, observes the outcomes of

those actions, and thereafter adjusts its policy to improve future performance its objective is to maximize cumulative rewards over time. Different reinforcement learning use cases call for different types of agents. In this implementation, a single agent is trained to play the game, and any other player involved is considered part of the environment of the game.

State

A state is a representation of the current situation of the environment [19]. It contains all the information necessary for the agent to make a decision. The state can be represented in various forms depending on the complexity and nature of the environment [20]. In the case of Oware, a possible state could be the array:

$$\begin{aligned} &[[4, 4, 4, 4, 4, 4], \\ &[4, 4, 4, 4, 4, 4]], \end{aligned}$$

This indicates the initial state of the game where all pits contain 4 seeds. Another possible state could be the array:

$$\begin{aligned} &[[4, 4, 4, 4, 4, 4, 0, 6], \\ &[4, 4, 4, 4, 4, 4, 0, 6]], \end{aligned}$$

where in this case, the store values and territory counts are added as additional information in the state vector.

Environment

The environment is everything outside the agent that the agent interacts with [19]. It responds to the agent's actions and provides rewards and new states. The environment can be modeled as a Markov Decision Process (MDP) [20]. In the game of Oware, the environment refers to the external system of the game with which the agent interacts, consisting of the game board, the rules, state transitions, and the opponent.

Markov Decision Process (MDP)

An MDP is a mathematical framework used in reinforcement learning to model decision-

making situations where outcomes are partly random and partly under the control of a decision-maker [5]. It is defined by the tuple (S, A, P, R, γ) [5]:

- S denotes the state space, or the space containing snapshots of the environment at different points in time.
- A denotes the action space, or space containing all the possible action choices that could be made by an agent that causes a change in the state of the environment.
- $P : S \times A \times S \rightarrow [0, 1]$ denotes the state transition probabilities, which define the probability of the environment changing from one state to another, as a result of an action.
- $R : S \times A \rightarrow \mathbf{R}$ is the reward function, which is what encourages or discourages certain actions from being taken by the agent at certain states of the environment.
- $\gamma \in (0, 1)$ is a ratio that measures the premium that the agent puts on expected future rewards, at the expense of current rewards. A higher γ places more importance on long-term rewards thus encouraging the agent to employ strategies that overall maximize future benefits.

Bellman's Equations Bellman's equations are central to many RL algorithms. They provide a recursive decomposition for the value function, which represents the expected cumulative reward an agent can obtain from any given state [5].

The value function $V^\pi(s)$ for a policy π is given by:

$$V^\pi(s) = \mathbf{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right],$$

where $V^\pi(s)$ represents the expected cumulative reward for an agent starting from state s and following policy π . Here, $\mathbf{E}_\pi$ denotes the expectation under the policy π , meaning that all possible sequences of states and actions that might be encountered by the agent when following the policy π are averaged. This involves the probabilistic nature of the environment, where transitions between states are governed by a probability distribution. The expectation accounts for all these possible transitions and actions to compute the average cumulative reward that can be expected starting from state s and adhering to policy π . $R(s_t, a_t)$ is the reward received at time t when in state s_t and taking action a_t , and γ is the discount factor which determines the importance of future rewards [5].

Bellman's optimality equation for the value function $V^*(s)$ is:

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V^*(s')],$$

which represents the maximum value achievable from state s by considering the best possible actions. In this equation, $P(s' | s, a)$ denotes the transition probability from state s to state s' given action a , and $V^*(s')$ is the optimal value function of the next state s' [5].

The Q-value function $Q^\pi(s, a)$ is defined as:

$$Q^\pi(s, a) = \mathbf{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a \right], \quad (1)$$

representing the expected cumulative reward of taking action a in state s and thereafter following policy π . This function evaluates the benefit of taking a specific action in a given state under a particular policy [5].

Bellman's optimality equation for the Q-value function $Q^*(s, a)$ is:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a'),$$

indicating the highest expected cumulative reward achievable by taking action a in state s and then following the optimal policy. This equation is used in Q-learning and other algorithms to iteratively improve the action-value estimates [21].

Policy-Based Learning

In policy-based learning, the agent learns a policy $\pi(a | s)$ directly, which maps states to actions. Policy gradient methods, such as REINFORCE, are common in this category. The policy is typically parameterized by θ , and the objective is to maximize the expected cumulative reward [22]:

$$J(\theta) = \mathbf{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right].$$

The gradient of the objective function concerning the policy parameters θ is given by [5]:

$$\nabla_\theta J(\theta) = \mathbf{E}_\pi [\nabla_\theta \log \pi_\theta(a_t | s_t) Q^\pi(s_t, a_t)].$$

Value-Based Learning

In value-based learning, the agent learns the value function or Q-value function [23]. Methods like Q-learning and SARSA fall into this category. The agent updates its estimates of the value functions based on the observed rewards and transitions.

The Q-learning update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

where s' is the next state resulting from taking the action a in state s , α is the learning rate, $R(s, a)$ is the reward received after taking action a in state s and γ is the discount factor. The term $\max_{a'} Q(s', a')$ represents the maximum estimated future reward attainable from the next state s' [5]. Here, the set a' includes all possible actions in the next state, s' .

SARSA (State-Action-Reward-State-Action) updates its estimates using the actual actions taken by the agent:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a) + \gamma Q(s', a') - Q(s, a)],$$

where a' is the action taken in state s' . This update rule uses the actual policy being followed by the agent [5].

Model-Based Learning

In model-based learning, the agent builds a model of the environment, including the transition probabilities $P(s' | s, a)$ and the reward function $R(s, a)$. The agent uses this model to plan and make decisions. Dynamic programming methods, such as value iteration and policy iteration, are examples of model-based learning [24].

Value iteration updates the value function using Bellman's optimality equation:

$$V(s) \leftarrow \max_a \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V(s')]$$

which represents the best possible value at state s , given all possible actions [5]. Policy iteration alternates between policy evaluation and policy improvement:

1. Policy evaluation:

$$V^\pi(s) = \sum_{s'} P(s' | s, \pi(s)) [R(s, \pi(s)) + \gamma V^\pi(s')],$$

where $\pi(s)$ is the policy's action for state s [5].

2. Policy improvement:

$$\pi(s) \leftarrow \arg \max_a \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V^\pi(s')].$$

This step updates the policy to be greedy concerning the current value function V^{pi} [5].

Model-Free Learning

In model-free learning, the agent does not build a model of the environment, but instead, it learns directly from experiences [25]. Temporal Difference (TD) learning is a popular model-free approach [26].

TD(0) learning updates the value function based on the difference between the predicted value and the actual reward received plus the discounted value of the next state [26]:

$$V(s) \leftarrow V(s) + \alpha [R(s, a) + \gamma V(s') - V(s)],$$

where α is the learning rate, $R(s, a)$ is the reward received for taking action a in state s , and $V(s')$ is the value of the next state s' [5]. This approach is used when the agent needs to update its value function based on the difference (or error) between predicted and actual outcomes.

After learning, the agent uses the learned policy to make decisions during the actual game or task. This is typically done by following the policy that maximizes the expected cumulative reward, as represented by the value function or Q -value function learned during training.

3 PROPOSED METHODS

DQNs combine Q-learning with deep neural networks to handle high-dimensional sensory inputs [11]. Key techniques such as experience replay and target networks are used to train and enhance performance [7]. They will be explained in this section. DDQNs are an extension of DQNs that address the overestimation bias which shows up in some use cases of DQNs. DDQNs aim to bring about more accurate value estimates when DQNs are not able to do so because of said overestimation bias [27].

The complex state space and strategic depth of Oware Nam-nam make these algorithms suitable for the game because of their ability to learn from high-dimensional data and adapt through experience [15].

3.1 Deep Q-Networks (DQN)

Deep Q-Networks (DQN) represent a breakthrough in reinforcement learning by combining Q-learning with deep neural networks. DQN is particularly notable for its success in achieving human-level performance on several Atari games from raw pixel inputs [7].

Q-Learning Recap: In the MDP formulation for RL, the process of learning to make decisions is driven by finding a policy that maximizes some expected cumulative reward function. This can be done either by using a model-based method, which involves constructing a model of the environment and using it to simulate and plan actions [5], or a model-free method, which involves learning a policy directly from interactions with the environment without explicitly modeling it [20]. In model-free methods, the agent learns to estimate the value of actions taken in different states, typically represented as Q-values, which help in making decisions that maximize cumulative rewards [28].

Q-learning is a model-free reinforcement learning algorithm to learn the value of taking an action in a particular state [23]. The Q -value update rule is defined in equation 2.3 above.

In DQN, a neural network is used to approximate the Q-value function $Q(s, a; \theta)$, where θ represents the weights and biases of the neural network [7]. The input to the network is the state, and the output is the Q-value for each possible action.

Experience Replay: Instead of learning from consecutive experiences, DQN stores ex-

periences (s, a, r, s') in a replay buffer [7]. During training, mini-batches of experiences are randomly sampled from the replay buffer to break the correlation between consecutive experiences and stabilize training.

Target Network: A separate target network $Q(s, a; \theta^-)$ with parameters θ^- is used to compute the target Q-value. The parameters θ^- are periodically updated to match the parameters θ of the primary Q-network.

Algorithm 1 DQN Training [7]

- 1: **Input:** Replay buffer D , Q-network parameters θ , target Q-network parameters θ^-
- 2: Initialize D to a fixed capacity
- 3: Initialize primary Q-network $Q(s, a; \theta)$ with random weights
- 4: Initialize target Q-network $Q(s, a; \theta^-)$ with weights $\theta^- = \theta$
- 5: **for** each episode **do**
- 6: Initialize state s
- 7: **for** each step in the episode **do**
- 8: Select action a using an ϵ -greedy policy derived from $Q(s, a; \theta)$
- 9: Execute action a , observe reward r and next state s'
- 10: Store experience (s, a, r, s') in D
- 11: Sample mini-batch of experiences (s_j, a_j, r_j, s'_j) from D
- 12: Compute target for each sampled experience:

$$y_j = \begin{cases} r_j & \text{if episode terminates at step } j + 1 \\ r_j + \gamma \max_{a'} Q(s'_j, a'; \theta^-) & \text{otherwise} \end{cases}$$

- 13: Perform gradient descent step on loss:

$$L(\theta) = \mathbf{E}_{(s_j, a_j, r_j, s'_j) \sim D} [(y_j - Q(s_j, a_j; \theta))^2]$$

- 14: **if** every C steps **then**
 - 15: Update target network: $\theta^- \leftarrow \theta$
 - 16: **end if**
 - 17: **end for**
 - 18: **end for**
-

3.2 Double Deep Q-Networks (DDQN)

Double Deep Q-Networks (DDQN) is an extension of DQN, which uses the Double-Q learning method introduced by Hado van Hasselt in 2010 [27]. The use of this method makes DDQN more accurate by using two sets of Q-values [27]. One set of Q-values is used to choose the best action, while the other set is used to evaluate that action. This helps

prevent mistakes in estimating the value of actions, thereby addressing the overestimation bias that often occurs in DQN. [27].

For DDQN, the primary Q-network $Q(s, a; \theta)$ selects the action, while the target Q-network $Q(s, a; \theta^-)$ evaluates the action.

Algorithm 2 DDQN Training Procedure [27]

```

1: Initialize replay buffer  $D$  to capacity
2: Initialize primary Q-network  $Q(s, a; \theta)$  with random weights
3: Initialize target Q-network  $Q(s, a; \theta^-)$  to match primary network ( $\theta^- = \theta$ )
4: for each episode do
5:   Initialize state  $s$ 
6:   for each step of the episode do
7:     Choose action  $a$  using  $\epsilon$ -greedy policy from  $Q(s, a; \theta)$ 
8:     Take action  $a$ , observe reward  $r$ , and next state  $s'$ 
9:     Store transition  $(s, a, r, s')$  in  $D$ 
10:    Sample random mini-batch from  $D$ :  $(s_j, a_j, r_j, s'_j)$ 
11:    Set  $y_j$  for each mini-batch sample:
12:    if episode terminates at  $j + 1$  then
13:       $y_j \leftarrow r_j$ 
14:    else
15:       $a^* \leftarrow \arg \max_{a'} Q(s'_j, a'; \theta)$ 
16:       $y_j \leftarrow r_j + \gamma Q(s'_j, a^*; \theta^-)$ 
17:    end if
18:    Perform gradient descent on  $(y_j - Q(s_j, a_j; \theta))^2$  with respect to  $\theta$ 
19:    if  $C$  steps have passed then
20:      Update target network:  $\theta^- \leftarrow \theta$ 
21:    end if
22:  end for
23: end for

```

DDQN generally leads to more stable and reliable learning compared to DQN [27].

4 EXPERIMENTS

4.1 Game Environment Setup

The entire project was carried out in Python and used libraries like NumPy, TensorFlow, Matplotlib, TensorBoard, and Tkinter. In this subsection, we consider the game environment setup.

MDP Formulation of the Game

States: The states of the game are the various configurations for the number of seeds in the pits of the board. Given that the total number of seeds is 48, the total number of pits is 12, and at any given point in the game, the total number of seeds on the board is a multiple of 4 between 0 and 48 (excluding 4), then the estimated size of the state space is very large, and this size of state space is one of the reasons why model-based reinforcement learning was not considered for this implementation of the game. The algorithm used to arrive at this value can be seen in the appendix.

Actions: In the game, the actions taken are according to the pits. Having represented the states as a 2D array, the action space would ideally be $(0, 0)$, $(0, 1)$, $(0, 2)$, $(0, 3)$, $(0, 4)$, $(0, 5)$, $(1, 0)$, $(1, 1)$, $(1, 2)$, $(1, 3)$, $(1, 4)$, $(1, 5)$. To simplify things, however, the action space is the list of integers from 0 to 11, where the mapping between the coordinates and integers is as follows:

$(0, 0) \rightarrow 0$	$(0, 1) \rightarrow 1$	$(0, 2) \rightarrow 2$	$(0, 3) \rightarrow 3$
$(0, 4) \rightarrow 4$	$(0, 5) \rightarrow 5$	$(1, 0) \rightarrow 6$	$(1, 1) \rightarrow 7$
$(1, 2) \rightarrow 8$	$(1, 3) \rightarrow 9$	$(1, 4) \rightarrow 10$	$(1, 5) \rightarrow 11$

Rewards: Rewards in the game are calculated based on the player's actions and the resulting changes in the game state [19]. The following are two methods for calculating rewards:

This first reward function for the DQN agent largely encourages the agent to capture seeds

Algorithm 3 Reward Calculation Method 1

```

1: Input: Player ID, current store state
2: Output: Reward
3: Initialize reward to 0
4: Calculate the number of seeds captured by the player
5: if seeds captured > 0 then
6:   Add CAPTURE_REWARD to the reward
7: end if
8: if player wins the game then
9:   Add GAME_WIN_REWARD to the reward
10: end if
11: return reward

```

and win the game by providing positive rewards for actions that led to that.

Specifically,

$CAPTURE_REWARD = 10 \times \text{number of seeds captured}$, for any seed capture, and
 $GAME_WIN_REWARD = 100$ for any game won.

The strength of this reward function is in its simplicity and direct encouragement of winning behaviors. However, the disadvantage is that it may not sufficiently penalize non-winning but valid moves, leading to suboptimal strategies that do not maximize long-term gains.

Algorithm 4 Reward Calculation Method 2

```

1: Input: Player, stores list, round winner, game-winner
2: Output: Reward
3: Initialize reward to 0
4: Calculate the number of seeds captured by the player
5: if seeds captured > 0 then
6:   Add CAPTURE_REWARD to the reward
7: else if seeds captured == 0 then
8:   Add NON_CAPTURE_PENALTY to the reward
9: end if
10: if player wins the round then
11:   Add ROUND_WIN_REWARD to the reward
12: else if player loses the round then
13:   Add ROUND_LOSS_PENALTY to the reward
14: end if
15: if player wins the game then
16:   Add GAME_WIN_REWARD to the reward
17: end if
18: return reward

```

The second reward function introduces more specifically defined incentives and disincentives.

tives. It rewards seed capture and round wins, penalizes non-capture and round losses, and provides a significant reward for winning the game.

Specifically,

$CAPTURE_REWARD = 40$ for any seed capture,

$ROUND_WIN_REWARD = 80$ for winning a round,

$GAME_WIN_REWARD = 700$ for winning the game,

$NON_CAPTURE_PENALTY = 0$ for capturing no seeds,

and $ROUND_LOSS_PENALTY = -1$ for losing a round.

This method encourages the agent to not only focus on winning but also to play strategically throughout the game. The advantage of this reward function is in the promotion of a more balanced strategy that considers both immediate and future rewards. However, this comes at a small cost of requiring more training time for the agent to learn effectively because of the added complexity of the rewards.

Structure of Game Environment

The game environment setup involves four main classes: `Board`, `RuleEngine`, `GameState`, and `GameController`. These classes ensure that all actions taken by the agent follow the game's rules, simplifying the training process.

The `Board` class is responsible for the initialization and the maintenance of the oware board. The board consists of two rows and six columns, which is represented as a 2 by 6 array, such that, the number of seeds in the pit is represented by the number in the index position of that pit in the array. Aside from the board, the other important aspects of the game managed by the board class are the stores, which keep track of the number of seeds captured by each player, and the territories which keep track of the number of territories possessed by players. The `RuleEngine` class contains the rules of the game and how actions are validly selected by players. It ensures that actions taken by players are legal according to the game rules, checks conditions to end a round or the game based on the current board state and player scores, and evaluates the reward for each action based on the game's rules and current state. The `GameState` class tracks the overall state of the game, including the board configuration, player scores, and the sequence of actions taken. It maintains the current state of the board, keeps a log of all actions taken during the game, and stores and updates the rewards for actions taken. This centralized state management allows for easy tracking and updating of the game's progress. The `Player` class handles player-specific actions and strategies. It manages the execution of player moves and interacts with the `GameState` to update the game accordingly. This class defines strategies for move selection, whether random or based on learned policies,

and performs actions chosen by the player. The `GameController` class orchestrates the flow of the game, managing rounds and player turns. It interacts with the `Board`, `GameState`, `Player`, and `RuleEngine` classes to execute the game logic. This class is responsible for setting up the game with the necessary components, alternating turns between players, executing their chosen actions, and updating the game state after each action. It contains the 2 main functions necessary for running the implementation of the game: `step`, `reset`.

4.2 Training

The development of the learning agent for Oware Nam-nam involves designing the neural network architecture, initializing the agent, implementing the learning algorithm, and integrating the agent with the game environment. The main agents considered for the thesis are Random Agent, DQN Agent, and DDQN Agent.

Network Architecture

Deep Q-Network

The deep Q-network used takes as input the state of the game, with a state size of 12, where 12 represents the number of pits in the oware board, and values represent the number of seeds in each pit. This can communicate to a large extent the condition of the game's environment at any given point in time. While the store values and territory count values could have been appended to the state vector, their addition did not seem to contribute much to the performance of the agent.

The network also contains 6 hidden layers, to ensure that the complexity in the relationships between the state vector and the value of actions is adequately captured. Each layer uses the same Rectified Linear Unit (ReLU) activation function to allow the model to establish non-linear relationships between the state vector and the -values for the actions and to control the problem of vanishing gradients, with the number of neurons used being 128. The weights and biases are also initialized using the Random Normal Initializer with a mean of 0.0 and a standard deviation of 0.05. A ridge regression regularization with a penalty of 0.01 is also used to control overfitting.

Additionally, the DQN architecture makes use of a memory buffer, which stores information about the experiences of the agent, specifically, the memory buffer contains tuples of

the current game state, action taken at the state, reward obtained from the action taken, and the next state resulting from it. This store of experiences is supposed to be pivotal for the agent's decision-making process since it helps in the value assignment of the value of an action at a given state. Because of the relative similarity of consecutive states, the size of the memory buffer is chosen to be the large value of 2000 to ensure that the agent is not limited in the experiences with which he uses to learn the q-values for actions. The buffer's main purpose is to facilitate the experience replay, and it's also important for enhancing the stability of the learning process. There is also a discount factor, represented by λ which regulates the amount of consideration the agent takes to the expected future returns of a given action at a given state. A higher gamma, generally means more consideration is given to future rewards of an action, rather than immediate ones, and for the agent, $\gamma = 0.99$. While a large γ slows down the convergence of the DQN agent to an optimal policy, as compared to lower values, over the lengthened period of the training, it leads to the DQN agent's convergence to better policies.

Another important hyperparameter set for the DQN agent was the ϵ parameter, which represented the exploration-exploitation ratio used by the agent in the choice of which action to take. For this game, the DQN agent was able to learn better with a lower value of ϵ , specifically 0.6, and a larger epsilon decay rate of 0.995. This is because a game is made of several rounds, where the rules of possible actions for a particular configuration of the board are modified based on the round and thus, the optimal action for a given state is dependent on not only the board configuration but also the round number and the player who won the round before. This means that an agent who learns that an action taken at a given board configuration yields a high reward is unlikely to get such a reward if he plays the same action in the same configuration but over a different round. This necessitates the agent to adopt a more random action choice approach for the majority of the earlier games it is trained on, to allow it to observe and learn the right q value for an action in a given state. The learning process is further stabilized by using the Adam optimizer with a scheduled learning rate ranging between the values: 0.00001, 0.0001, and 0.0005 and gradient clipping to prevent exploding gradients, ensuring smooth and stable training.

The architecture also included a target network, which is updated at a lesser frequency compared to the primary network being used to predict the q values for the actions. For this network, the update is done every 10 epochs, and its purpose is to improve the performance of the DQN by reducing instability and divergence which DQN is mostly prone to especially because of the presence of similar experiences in the memory buffer and the overestimation of q values of actions among other reasons.

Double Deep Q-Networks

The Double Deep Q-Network (DDQN) used in this implementation takes as input the state of the game, with a state size of 12, representing the number of pits on the Oware board, where the values indicate the number of seeds in each pit. This input effectively communicates the condition of the game's environment at any given point in time. Although the store values and territory count values could have been appended to the state vector, their inclusion did not significantly enhance the agent's performance.

The network includes 6 hidden layers to capture the complex relationships between the state vector and the value of actions. Each layer employs the Rectified Linear Unit (ReLU) activation function, which allows the model to establish non-linear relationships between the state vector and the Q-values for the actions while mitigating the problem of vanishing gradients. Each layer contains 128 neurons. The weights and biases in these layers are initialized using the Random Normal Initializer with a mean of 0.0 and a standard deviation of 0.5. To control overfitting, ridge regression regularization with a penalty of 0.01 is applied.

Additionally, the DDQN architecture utilizes a memory buffer that stores information about the agent's experiences, which are represented as tuples of the current game state, the action taken by the agent at that state, the reward obtained from the action, and the next state resulting from that action. These stored experiences are what the agent draws on for decision-making purposes since they help in determining the value of an action at a given state. Due to the relative similarity of consecutive states, the memory buffer size is set to 2000 to ensure that the agent has access to a wide range of experiences, facilitating the learning of Q-values for actions. The primary purpose of the buffer is to support experience replay, enhancing the stability of the learning process.

The discount factor, represented by γ , regulates the consideration the agent gives to the expected future returns of a given action at a given state. A higher γ , set at 0.9 for this agent, means more consideration is given to future rewards rather than immediate ones. While a large γ can slow down the convergence of the DDQN agent to an optimal policy, over a prolonged training period, it leads to convergence on better policies.

Another important hyperparameter is the ϵ parameter, which represents the exploration-exploitation ratio used by the agent when choosing actions. For this game, the DDQN agent begins with an epsilon value of 1.0 and decays it with a rate of 0.995. The learning process is further stabilized by using the Adam optimizer with a learning rate of 0.01

and gradient clipping, preventing exploding gradients and ensuring smooth and stable training.

The architecture also includes a target network, which is updated less frequently compared to the primary network used for predicting Q-values for actions. In this implementation, the target network is updated every 10 epoch. The inclusion of a target network helps improve the performance of the DDQN by reducing instability and divergence, common issues in DQN due to the presence of similar experiences in the memory buffer, and the overestimation of Q-values among other reasons.

Comparative Analysis

Given the nuances of the game, specifically the several rounds of play that make up the game and how the rules of picking actions, and the benefits of those actions change from round to round, the deep Q-network has a higher propensity to diverge, and therefore the loss values obtained during training are very volatile with a very slight downward direction. The DDQN, however, is better able to navigate this problem, giving it a generally better result in playing the game. This is because its target network serves as a form of predictor-corrector, helping the network to control the magnitude of q-values that are assigned to actions.

For both models, the hyperparameters used were mostly similar to ensure a fair comparison of their performances. A batch size of 74 was used to ensure that for each step during training the agent had enough learning experiences to use in their assignment of Q-values to actions, and the choice of actions to take. It was generally observed that although larger batch sizes increased training time significantly, the agent had a better chance of higher performance, owing to a larger amount of experience to learn from. A batch size lower than 50 has little to no hope of producing good results against a random agent because the minimum number of turns that make up a game is around 60, implying that if an agent is using 50, a good picture of the actions in the game and their corresponding consequences is not available for the agent to use and thus the experience the agent draws from is biased and unreliable. Learning rates for both agents were chosen to be very low, as the several occurrences of the same board configurations for which particular actions had varying Q-values meant that the agent did not do well when it made quick decisions on the Q-value to assign to actions. Overall, larger learning rates produced exploding loss values for both agents.

Training Procedures

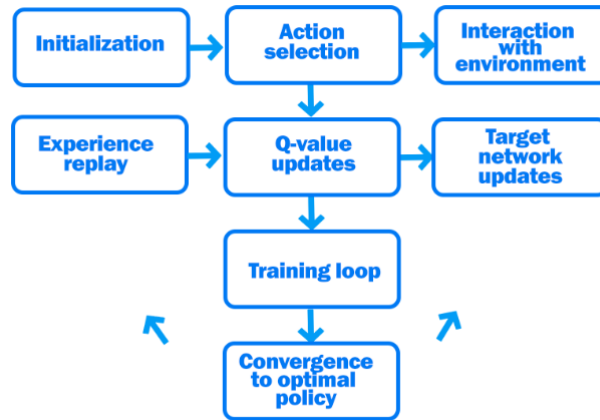


Figure 4. DQN algorithm flow diagram

DQN Training Algorithm

The DQN training algorithm can be summarized in the diagram 4.

From the figure above, we observe that the deep Q-network process starts with an initialization of the Q-network with random weights, which in this implementation was done using the random normal distribution sampling with a mean of 0 and standard deviation of 0.05. The agent starts to play the first "batch_size" number of games by randomly selecting the actions and playing them. This allows for the population of the memory buffer, which is then used to allow the agent to select actions based on experience, by evaluating the Q-values of these actions using the Q-network. The epsilon value determines at each point after the first games are played, whether the agent should pick an action randomly, or based on the experience gathered. The Q-values are particularly updated in the following way. When the state, action, reward, and next state tuples are saved in the memory buffer, and it is time for the agent to take an action, the agent samples a batch of these samples, with a size of the batch size, and for each tuple, calculates the target Q-value of the given action at the given state, using the Bellman equation as represented in 1. The agent then calculates the loss as the mean squared error between the target Q-value and the Q-value predicted by the Q-network for the given action-state pair. The loss is then minimized, with the help of the gradient descent step thus leading to the update of the weight and biases in the Q-network [7].

DDQN Training Algorithm

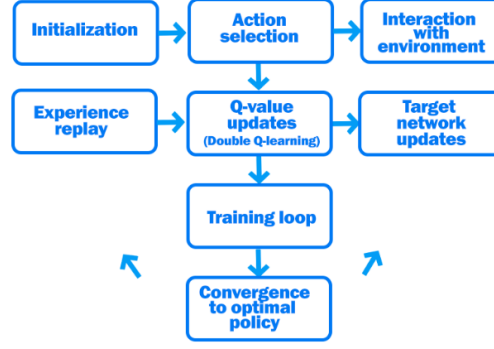


Figure 5. DDQN algorithm flow diagram

Similar to the DQN algorithm employed above, the DDQN training algorithm can also be summarized in the diagram 5.

While this is very similar to the DQN, the fundamental difference comes where the target Q-value is calculated. In the DDQN approach, instead of using the maximum estimated Q-value for the next state as in DQN, the target Q-value is calculated using the action that the current Q-network would choose. Specifically, the DDQN decouples the selection and evaluation of the action, which helps to reduce the overestimation bias observed in DQN. The target Q-value in DDQN is calculated as:

$$Q_{\text{target}} = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$$

where r is the reward, γ is the discount factor, s' is the next state, θ are the weights of the Q-network, and θ^- are the weights of the target network. This method helps in providing a more accurate estimate of the Q-values, thereby improving the stability and performance of the learning process. The rest of the training procedure, including the initialization, memory buffer population, epsilon-greedy policy for action selection, and loss minimization through gradient descent, remains the same as in the DQN algorithm.

Training Metrics and Curves

Training Metrics

The main metrics used to observe the training process of the agents were loss, reward, and epsilon. These metrics, which would be explained below are enough to visualize the quality and stability of the learning process of the agent.

The loss metric measures the amount of deviation of the agent-predicted valuation of an action taken at a given state from the target valuation or target Q-value of that action. A large loss means the agent has not succeeded in determining the most profitable actions to take at the different states it encounters during gameplay, and it is therefore either assigning too high Q-values to actions with low target Q-values, or too low Q-values to actions with high target Q-values. An unstable loss function also is a sign that the learning rate might be too high, the batch size or memory buffer might be too small or not diverse enough or the agent has not had enough time to explore and is therefore making decisions based on biased experiences. Spikes in the loss function are however quite normal in the beginning stages of training, when the agent is just exploring new actions.

The reward metric shows us how well the agent is doing in increasing the level of accumulated rewards over the multiple games it plays during the training. The reward, whether cumulative or average is generally expected to either grow or increase in frequency with more training because it is expected that as the agent learns and improves its policies, it will make better decisions, leading to higher rewards. Rewards are central to the learning process as they teach the agent how to behave. They are the main component of the Q-value assigned to actions. A reward that is not improving as expected could indicate that it's time for the agent's learning or exploration strategy to be reconsidered.

The epsilon decay, in conjunction with the reward and loss metrics, helps to identify whether certain inconsistencies in the direction of motion in the loss or reward graph might be due to insufficient exploration or too much exploration by the agent.

For the training of DQN and DDQN agents for gameplay, the training was done in 2 main generations, wherein in the first generation, the agent was trained with a random player as its opponent, and in the second generation, the agent was trained with a previous generation player as its opponent. This was done to further encourage the model to perform better and to see how well the model could do.

DQN Training Metrics: In the training of the agents, a first round of training was conducted between an agent (DQN or DDQN) and a random player, and the agents resulting from that training are referred to as first-generation agents in this thesis. The second generation agents are those agents who have been trained against the first generation agents and therefore are superior to the first generation agents.

For the first-generation training against the random player, the metrics highlighted earlier are seen here.

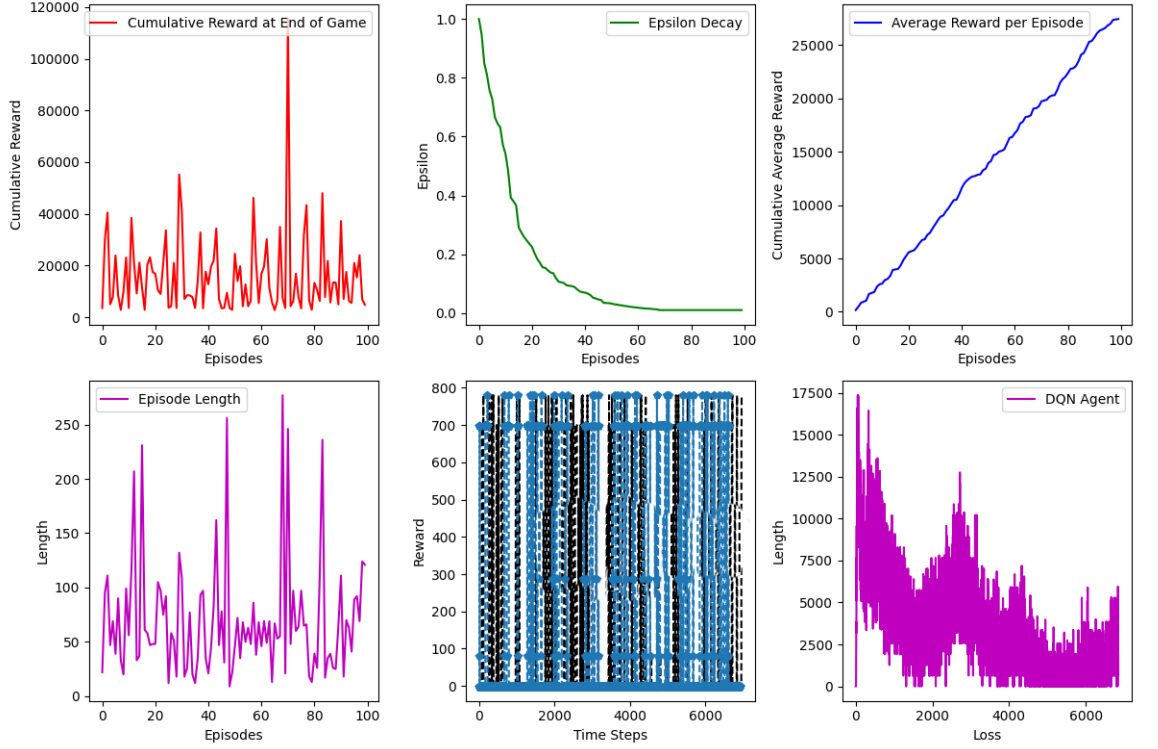


Figure 6. Training Statistics for DQN Agent

For the training statistics of DQN 6, no visible trend is seen in the cumulative reward at the end of each game, however, we observe that the cumulative average reward after each epoch increases steadily, hinting that there is some continuous improvement in how the agent makes decisions over the rounds. From the reward function 4, we see that the agent is rewarded for capturing seeds, which happens more consistently as evidenced by the cumulative average reward graph. The episode length, much like the reward at the end of each game, shows no clear trend, despite the penalty of not winning a round, which indicates a probable necessary increase in the penalty or a change of the structure of the reward function altogether. The reward plot shows the amount of rewards obtained by the agent per turn and gives a picture of how the game went throughout training. The final graph shows the loss function, and we observe that the loss has a highly volatile downward trajectory over the training period.

A second-generation model was trained based on the first one, and the results will be considered in more detail in section 5.

DDQN Training Metrics: For the first-generation training against the random player, the metrics highlighted earlier are seen here.

For the training statistics of the DDQN agent, the cumulative reward at the end of each

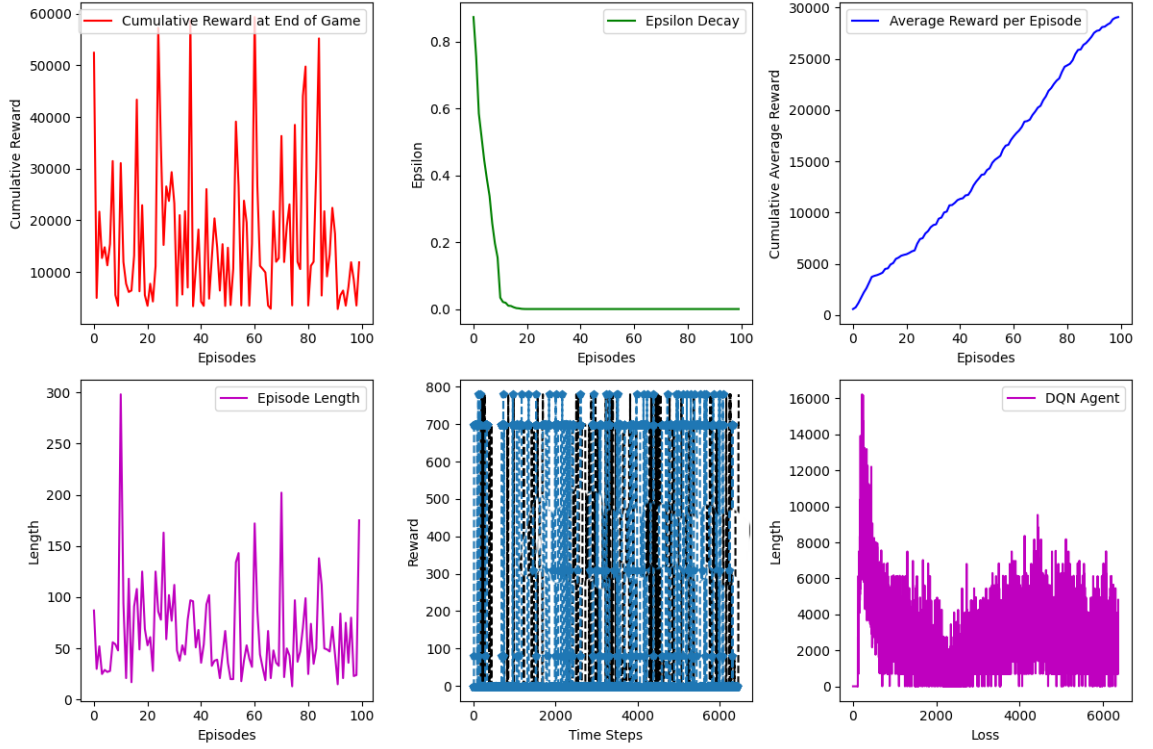


Figure 7. Training Statistics for DDQN Agent

game shows considerable fluctuations, similar to the DQN agent. However, the cumulative average reward increases steadily, indicating a consistent improvement in the agent’s decision-making process throughout the training. The rapid epsilon decay in the DDQN agent ensures that it transitions quickly to exploiting its learned policies, allowing it to consolidate its strategies early on. This early transition to exploitation, despite causing variability in episode lengths, helps the agent focus on refining high-reward strategies, as reflected in the peaks of cumulative rewards.

The DDQN agent’s variability in episode lengths suggests that while it occasionally finds highly rewarding strategies, it also encounters inconsistent performance, likely due to its complex target update mechanism that aims to reduce overestimation bias. This mechanism, although beneficial in reducing bias, introduces some instability, leading to the observed variability.

The dense clustering of rewards for the DDQN agent indicates that it frequently encounters and effectively navigates similar states, applying learned strategies to achieve high rewards. The loss function for the DDQN agent, much like the DQN agent, shows a highly volatile trajectory, reflecting the dynamic and ongoing learning process. This fluctuation in loss signifies the agent’s continuous effort to balance exploration and exploitation,

adapting its strategies to maximize rewards.

Both DQN and DDQN agents demonstrate strong learning capabilities and continuous improvement in their decision-making processes. To further enhance their performance, gradual fine-tuning of the learning rate, a more moderated epsilon decay, and incorporation of regularization techniques can be implemented. Ensuring a diverse replay buffer and adjusting the batch size can also help stabilize the learning process, leading to more consistent and reliable results. By focusing on these adjustments, both agents can better explore and exploit the game's strategies, ultimately improving their overall performance.

4.3 Testing and Evaluation

Evaluation Setup: To evaluate the models, a comprehensive test was conducted between random, DQN, and DDQN agents. The evaluation aimed to determine the best-performing model based on several metrics, including win rate and average reward per episode.

Testing Procedure: For the first-generation models of the DQN and DDQN agents, a test was run by using the weights learned during the training process to define the policy network used by the agent to play the game.

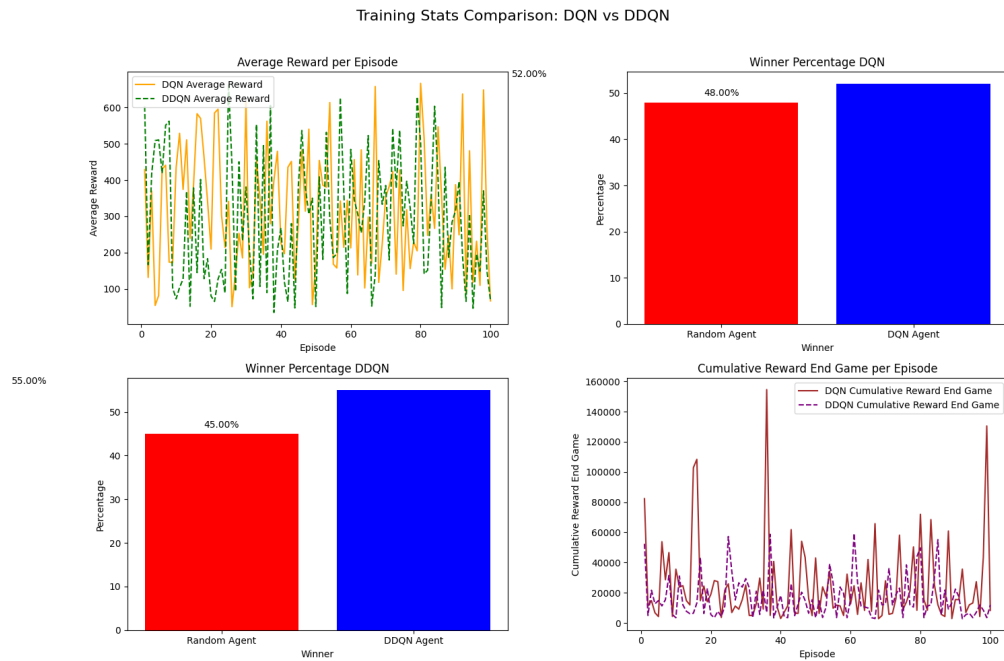


Figure 8. Test Statistics for DQN and DDQN Agents

The results showed that while the DDQN agent sometimes outperformed both random and DQN agents in terms of win rate and average reward per episode, its performance was not consistently superior. The DDQN agent's learning stability and bias reduction need more thorough examination and improvement.

The second generation model however showed a higher level of ability for both DQN and DDQN agents as compared to the random player. The results can be seen in figure 9.

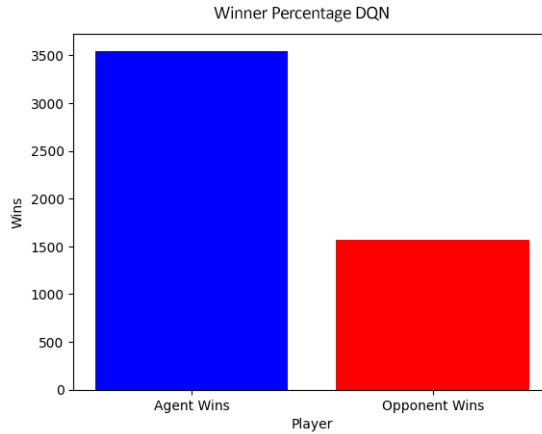


Figure 9. 2nd generation DQN agent vs random agent

From the graph in figure 9, we observe that the model performs much better than the first-generation model, with a win rate of 70% against the random agent.

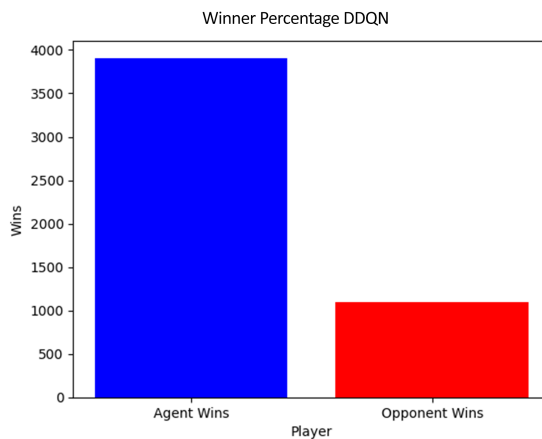


Figure 10. 2nd generation DQN agent vs random agent

As compared to the first-generation DQN and DDQN agents, as well as the second-generation DQN agent, the second-generation DDQN agent won with a win rate of about 80% against the random agent.

5 CONCLUSION

The results indicate that the DDQN agent performed much better than the DQN and random agents. The separation of action selection and evaluation in DDQN was, therefore, an important element of the agent’s ability to learn how to play the game, as it effectively reduced the overestimation bias which in turn led to more stable and accurate value estimates. The parameter sensitivity analysis showed how significant it was to choose the right hyperparameters and just how badly learning could be affected if it was not done well enough. The DDQN agent demonstrated that reinforcement learning could be effectively applied to the game, and achieve a considerable win rate, despite the game’s seeming random and wandering nature, thus giving hope about the possibility of developing an agent that could master the game and learn the best moves to consistently win. The implementation of DQN and DDQN agents for playing Oware Nam-nam and the evaluation of their performance shows the power of the double q-learning approach, and the impact of reward shaping as well as choosing a suitable ratio for exploitation and exploration of experience replay depending on the nature and nuances of the use case being considered.

Future work could enhance the neural network architecture even further to explore more complex ones, and to try to use prioritized experience replay to speed up the winning of a game for agents while maintaining or even increasing winning rates as well. Other reinforcement learning techniques, such as Proximal Policy Optimization (PPO) [29] or Advantage Actor-Critic (A2C) [30] could also be considered for enhancing the agent’s performance. The agents could also benefit even further from being trained over many more generations and being trained against agents learning using various reinforcement learning algorithms to increase knowledge about how these algorithms go about handling the nuances and challenges that come with the Oware game. Finally, even more sophisticated approaches such as multi-agent reinforcement learning could be applied to simulate more realistic game scenarios, allowing multiple agents to learn during gameplay, instead of effectively considering the opponent as part of the environment as was done in this work.

This study successfully demonstrated the application of reinforcement learning to the Oware Nam-nam game. The DDQN agent outperformed the DQN and random agents, achieving higher win rates and more stable learning. The parameter sensitivity analysis provided valuable insights into the impact of hyperparameters on agent performance. The study’s key contributions include the development and comprehensive evaluation of DQN and DDQN agents for Oware Nam-nam and insight into how these methods handle a game

with a high level of seeming randomness.

REFERENCES

- [1] Wikipedia contributors. Oware. <https://en.wikipedia.org/wiki/Oware>, 2021. Retrieved [5th June 2024].
- [2] The Oware Society. The Oware Society, 1994. Retrieved [29 May, 2024].
- [3] Africa Heartwood Project. Oware Mancala Seed Game. <https://africaheartwoodproject.org/product/oware-mancala-seed-game/>, n.d. Retrieved [6th June 2024].
- [4] Erdem Emekligil and Ethem Alpaydin. What’s in a game? the effect of game complexity on deep reinforcement learning. In *Computer Games: 7th Workshop, CGW 2018, Held in Conjunction with the 27th International Conference on Artificial Intelligence, IJCAI 2018, Stockholm, Sweden, July 13, 2018, Revised Selected Papers 7*, pages 147–163. Springer, 2019.
- [5] Sutton, Richard S. and Barto, Andrew G. *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- [6] Rothmann, Marc and Porrmann, Mario. A Survey of Domain-Specific Architectures for Reinforcement Learning. *IEEE Access*, 10:13753–13767, 2022.
- [7] Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Rusu, Andrei A., Veness, Joel, Bellemare, Marc G., Graves, Alex, Riedmiller, Martin, Fidjeland, Andreas K., Ostrovski, Georg, et al. Human-Level Control Through Deep Reinforcement Learning. *Nature*, 518(7540):529–533, 2015.
- [8] Mohamed, Abdel-rahman, Dahl, George, and Hinton, Geoffrey. Deep Belief Networks for Phone Recognition. In *NIPS Workshop on Deep Learning for Speech Recognition and Related Applications*, volume 1, page 39. NIPS Workshop, 2009.
- [9] Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey E. Imagenet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, 25, 2012.
- [10] Das, Arghyadeep, Shroff, Vedant, Jain, Avi, and Sharma, Grishma. Knowledge Transfer Between Similar Atari Games Using Deep Q-Networks to Improve Performance. In *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–8. IEEE, 2021.
- [11] Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Graves, Alex, Antonoglou, Ioannis, Wierstra, Daan, and Riedmiller, Martin. Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*, 2013.

- [12] Tesauro, Gerald. Temporal Difference Learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995.
- [13] Kaiser, Lukasz, Babaeizadeh, Mohammad, Milos, Piotr, Osinski, Blazej, Campbell, Roy H., Czechowski, Konrad, Erhan, Dumitru, Finn, Chelsea, Kozakowski, Piotr, Levine, Sergey, et al. Model-Based Reinforcement Learning for Atari. *arXiv preprint arXiv:1903.00374*, 2019.
- [14] Silver, David, Huang, Aja, Maddison, Chris J., Guez, Arthur, Sifre, Laurent, van den Driessche, George, Schrittwieser, Julian, Antonoglou, Ioannis, Panneershelvam, Veda, Lanctot, Marc, et al. Mastering the Game of Go with Deep Neural Networks and Tree Search. *Nature*, 529(7587):484–489, 2016.
- [15] Silver, David, Hubert, Thomas, Schrittwieser, Julian, Antonoglou, Ioannis, Lai, Matthew, Guez, Arthur, Lanctot, Marc, Sifre, Laurent, Kumaran, Dharshan, Graepel, Thore, et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- [16] Bai, Yu and Jin, Chi. Provable Self-Play Algorithms for Competitive Reinforcement Learning. In *International Conference on Machine Learning*, pages 551–560. PMLR, 2020.
- [17] Zhang, Hongming and Yu, Tianyang. AlphaZero. *Deep Reinforcement Learning: Fundamentals, Research and Applications*, pages 391–415, 2020.
- [18] Quansah, Emmanuel and Ross, Peter. Mancala Games: The Oware Game and Its Heuristics. *IEEE Transactions on Computational Intelligence and AI in Games*, 8(3):233–242, 2016.
- [19] Yuxi Li. Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274*, 2017.
- [20] Kaelbling, Leslie Pack, Littman, Michael L., and Moore, Andrew W. Reinforcement Learning: A Survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996.
- [21] Bertsekas, Dimitri. *Reinforcement Learning and Optimal Control*, volume 1. Athena Scientific, 2019.
- [22] Hugging Face. Policy Gradient with PyTorch. <https://huggingface.co/blog/deep-rl-pg>. Retrieved [6th June 2024].
- [23] Watkins, Christopher J.C.H. and Dayan, Peter. Q-Learning. *Machine Learning*, 8:279–292, 1992.

- [24] Rolf A. N. Starre, Marco Loog, Elena Congeduti, and Frans A. Oliehoek. An analysis of model-based reinforcement learning from abstracted observations, 2023.
- [25] Sinan Çalışır and Meltem Kurt Pehlivanoglu. Model-free reinforcement learning algorithms: A survey. In *2019 27th signal processing and communications applications conference (SIU)*, pages 1–4. IEEE, 2019.
- [26] Gerald Tesauro. Practical issues in temporal difference learning. *Advances in neural information processing systems*, 4, 1991.
- [27] Hasselt, Hado. Double Q-Learning. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23, 2010.
- [28] Feinberg, Vladimir, Wan, Alvin, Stoica, Ion, Jordan, Michael I., Gonzalez, Joseph E., and Levine, Sergey. Model-Based Value Estimation for Efficient Model-Free Reinforcement Learning. *arXiv preprint arXiv:1803.00101*, 2018.
- [29] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [30] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PMLR, 2016.

Appendix 0.

This appendix provides detailed information on the workflow and logic used in the project implementation. It includes visual representations and an algorithm to estimate the size of the state space.

0.1 Workflow Overview

The workflow for implementing this project and the structure of this section are summarized in Figure A0.11. This flowchart outlines the main steps and processes involved in the development and execution of the project.

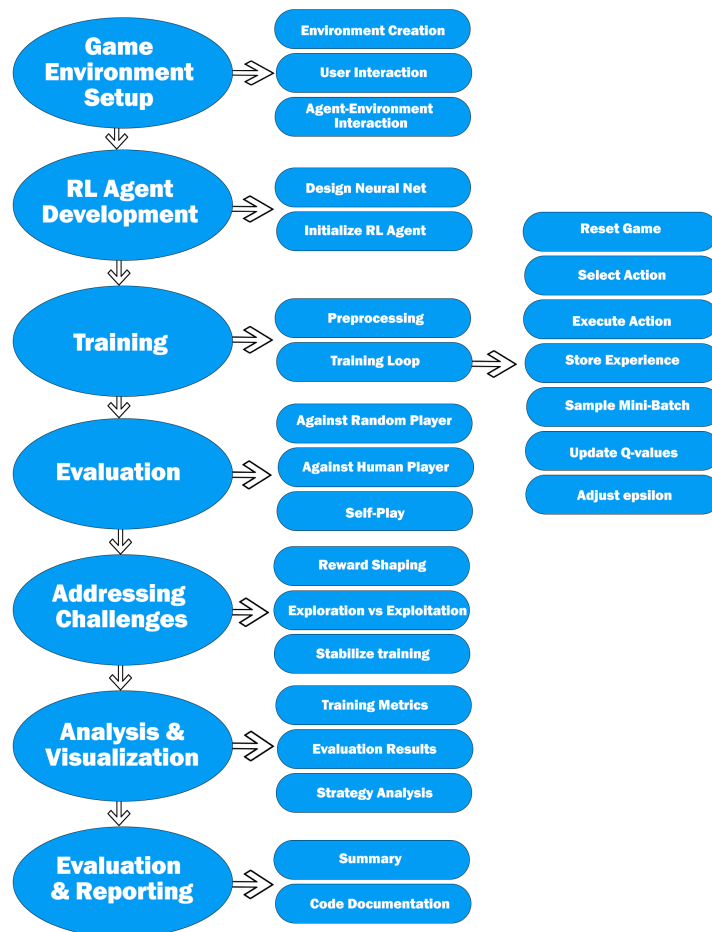


Figure A0.11. Operational flow

Appendix 0.

0.2 Game Logic Implementation

Figure A0.12 illustrates the simulation of a single turn in the Oware Nam-nam game, highlighting the sequence of actions and interactions between game elements.

```
START ROUND 1

New Board:
[[4 4 4 4 4 4]
 [4 4 4 4 4 4]]
Player 1 starts this round
Current player: 1
Player 1 action options [0, 1, 2, 3, 4, 5]

Player 1 chooses action: 0

CALCULATING REWARD ...

GAME STATE SAVING ...
[[6 1 6 1 7 2]
 [6 6 0 1 6 6]]
GAME STATE SAVED ...
Total states saved (2)

Current stores state:
[0 0]
Current territory count:
[6 6]
Switch Players ...
Current player: 2
Player 2 action options [6, 7, 9, 10, 11]

Player 2 chooses action: 10

GAME STATE SAVING ...
[[8 1 8 3 0 1]
 [1 8 2 3 0 9]]
GAME STATE SAVED ...
```

Figure A0.12. Oware Nam-nam game simulation of a single turn

Appendix 0.

0.3 State Space Calculation

To obtain a clearer picture of the game's complexity and motivate the RL algorithms used in the thesis, the size of the state space is estimated as follows in algorithm 5.

Algorithm 5 Calculate Size of State Space

```
1: Input: Total number of seeds  $T = 48$ , number of pits  $n = 12$ 
2: Output: Estimated size of the state space
3: state_space_size  $\leftarrow 0$ 
4: for each  $S \in \{0, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48\}$  do
5:   if  $S = 4$  then
6:     continue
7:   end if
8:   max_seeds_per_pit  $\lfloor \frac{S}{3} \rfloor$ 
9:   configurations  $\leftarrow 0$ 
10:  for  $i = 0$  to  $S$  do
11:    if  $i \leq \text{max\_seeds\_per\_pit}$  then
12:      configurations  $\leftarrow \text{configurations} + \binom{S+11-i}{11}$ 
13:    end if
14:  end for
15:  state_space_size  $\leftarrow \text{state\_space\_size} + \text{configurations}$ 
16: end for
17: return state_space_size
```
